

7_Sets in Python Methods and Manipulations

A set is an **unordered** and **mutable** collection of unique elements

```
myset={1,2,3,4,5}
print(type(myset))
```

[2] ✓ 0.0s

... <class 'set'>

```
another_set={3,3,4,5,6,7,8,9} #even though 3 is repeated it will be stored only once in the set
print(another_set)
```

[4] ✓ 0.0s

... {3, 4, 5, 6, 7, 8, 9}

```
# sets are unordered and no indexing is possible
print(another_set[0])
```

[6] ✗ 0.0s

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_2521/3518211398.py in <cell line: 0>()
      1 # sets are unordered and no indexing is possible
----> 2 print(another_set[0])
```

TypeError: 'set' object is not subscriptable

```
another_set.add(6) # adding an element to the set
print(another_set)
another_set.remove(6) # removing an element from the set
print(another_set)
another_set.discard(7) # removing an element from the set, but if the element is not present it will not throw an error
print(another_set)
another_set.discard(10) # removing an element from the set, but if the element is not present it will not throw an error
print(another_set)
another_set.pop() # removing a random element from the set
print(another_set)
another_set.update({10,11,12}) # adding multiple (set) elements to the set
print(another_set)
another_set.update([98,99,100]) # adding multiple (list) elements to the set
print(another_set)
```

[7] ✓ 0.0s

```
{3, 4, 5, 6, 7, 8, 9}
{3, 4, 5, 7, 8, 9}
{3, 4, 5, 8, 9}
{3, 4, 5, 8, 9}
{4, 5, 8, 9}
{4, 5, 8, 9, 10, 11, 12}
{4, 5, 8, 9, 10, 11, 12, 98, 99, 100}
```

```
#Basically discard() and remove() are used to remove elements from a set.
# The difference between the two is that discard() will not raise an error if the element is not found in the set
# meanwhile remove() will raise a KeyError if the element is not found.
```

[9] ✓ 0.2s



```
print(another_set.pop()) # removing a random element from the set
```

[10] ✓ 0.0s

... 5



```
# Union, Intersection, Difference, Symmetric Difference
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
print(set1.union(set2)) # Union of two sets
print(set1.intersection(set2)) # Intersection of two sets
print(set1.difference(set2)) # Difference of two sets
print(set2.difference(set1)) # Difference of two sets
print(set1.symmetric_difference(set2)) # Symmetric Difference of two sets
print(set1)
print(set2)
print(set1.difference_update(set2)) # Difference Update will bring the difference of two sets and update/overwrite the first set with the difference value
print(set1)
print(set2)
set3 = set1 - set2 # Difference of two sets but this will not update/overwrite the first set with the difference value
print(set3)
print(set1)
print(set2)
print(set1.intersection_update(set2)) # Intersection Update will bring the intersection of two sets and update/overwrite the first set with the intersection value
print(set1) # No matches between set1 and set2 so set1 will be empty
# | represents union, & represents intersection, - represents difference, ^ represents symmetric difference
new_set={3,4}
print(set2.issubset(new_set)) # check if set2 is a subset of new_set
print(new_set.issubset(set2)) # check if new_set is a subset of set2
new_set2=new_set.copy() # copy a set
print(new_set2)
```

[27] ✓ 0.0s

```
... {1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{5, 6}
{1, 2, 5, 6}
{1, 2, 3, 4}
{3, 4, 5, 6}
None
{1, 2}
{3, 4, 5, 6}
{1, 2}
{1, 2}
{3, 4, 5, 6}
None
set()
False
True
{3, 4}
```